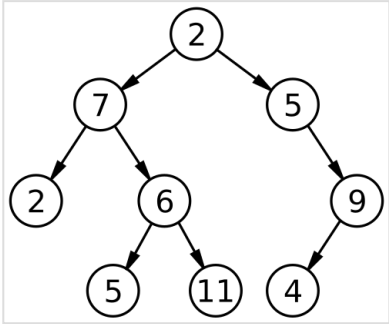
维基百科
自由的百科全书

二叉树

在计算机科学中，二叉树（英语：Binary tree）是每个节点最多只有两个分支（即不存在分支度大于2的节点）的树结构[1]。通常分支被称作“左子树”或“右子树”。二叉树的分支具有左右次序，不能随意颠倒。

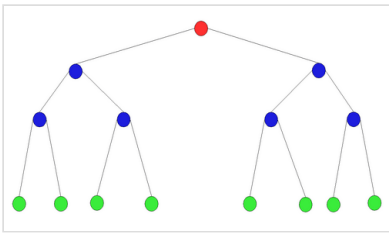


一棵有9个节点且深度为3的二叉树，其根节点的值为2，它既不平衡亦未经过排序

二叉树的第i层至多拥有2^{i-1}个节点；深度为k的二叉树至多总共有$2^{k+1}-1$个节点（定义根节点所在深度$k_0=0$），而总计拥有节点数符合的，称为“完美二叉树”；深度为k有n个节点的二叉树，当且仅当其中的每一节点，都可以和深度k的满二叉树，序号1到n的节点一对一对应时，称为“完全二叉树”。

与普通树不同，普通树的节点个数至少为1，而二叉树的节点个数可以为0；普通树节点的最大分支度没有限制，而二叉树节点的最大分支度为2；普通树的节点无左、右次序之分，而二叉树的节点有左、右次序之分。

二叉树通常作为数据结构应用，典型用法是对节点定义一个标记函数，将一些值与每个节点相关系。这样标记的二叉树就可以实现二叉搜索树和二叉堆，并应用于高效率的搜索和排序。

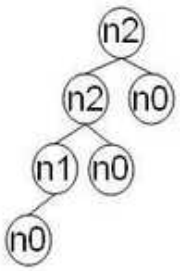

一棵简单的完美二叉树

图论中的定义

二叉树是一个连通的无环图，并且每一个顶点的度不大于3。有根二叉树还要满足根节点的度不大于2。有了根节点之后，每个顶点定义了唯一的父节点，和最多2个子节点。然而，没有足够的信息来区分左节点和右节点。如果不考虑连通性，允许图中有多个连通分量，这样的结构叫做森林。

性质

二叉树是一个有根树，并且每个节点最多有2个子节点。非空的二叉树，若树叶总数为n_0，分支度为2的总数为n_2，则$n_0=n_2+1$。

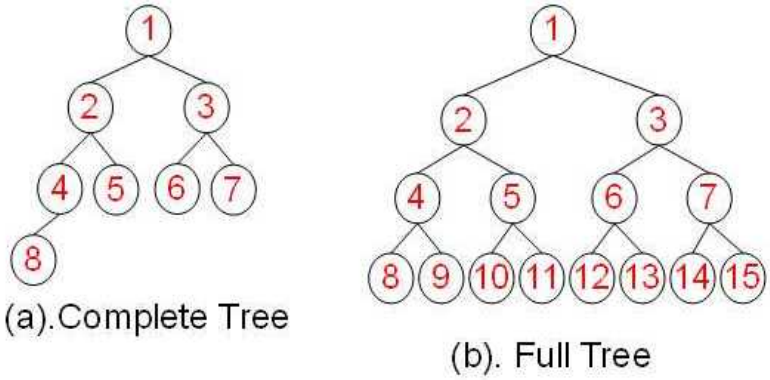


n0 = 3 (leaf/node)
n1 = 1
n2 = 2 (full node)
e = 5
n = 6

n = n0 + n1 + n2
n = e + 1
e = n1 + 2n2

n0 = n2 + 1

特殊类型



	(a)完全二叉树	(b)完美二叉树
总节点k	$2^{h-1} \leq k \leq 2^h - 1$	$k = 2^h - 1$
树高h	$h = log_2 k + 1$	$h = log_2(k+1)$

满二叉树

在一棵二叉树中，每个节点都要么有0个子节点，要么有2个子节点，则这种二叉树称作满二叉树（Full Binary Tree）。

满二叉树亦可通过如下递归定义得到：

- 仅有单个节点（根节点）的二叉树是满二叉树；
- 根节点有2个子树且均为满二叉树的二叉树是满二叉树。

完美二叉树

一棵深度为k，且有$2^k - 1$个节点的二叉树，称为完美二叉树（Perfect Binary Tree）。这种树的特点是其中所有内部节点都有两个子节点，并且所有叶子都处于同一深度。

性质

对于一棵深度为 k 的完美二叉树：

- 共有 $2^{k+1}-1$ 个结点
- 结点个数一定为奇数
- 第 i 层有 2^{i-1} 个结点
- 有 2^k 个叶子

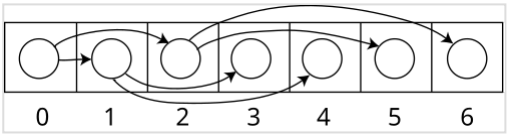
完全二叉树

在一颗二叉树中，若除最后一层外的其余层都是满的，并且最后一层要么是满的，要么在右边缺少连续若干节点，则此二叉树为完全二叉树（Complete Binary Tree）[2]。具有n个节点的完全二叉树的深度为$log_2 n+1$。深度为k的完全二叉树，至少有2^{k-1}个节点，至多有2^k-1个节点。

存储

在程序设计语言中能用多种方法来构造二叉树。

顺序存储表示



一个存储在数组中的完全二叉树

二叉树可以用数组或链表来存储，若是满二叉树就能紧凑排列而不浪费空间。如果某个节点的索引为i，（假设根节点的索引为0）则在它左子节点的索引会是$2i+1$，以及右子节点会是$2i+2$；而它的父节点（如果有）索引则为$\left\lfloor \frac{i-1}{2} \right\rfloor$。这种方法更有利于紧凑存储和更好的访问的局部性，特别是在前序遍历中。然而，它需要连续的存储空间，这样在存储高度为h的n个节点所组成的一般树时，将浪费很多空间。在最糟糕的情况下，如果深度为h的二叉树其每个节点都只有右孩子，则该存储结构需要占用2^h-1的空间，实际上却只有h个节点，浪费了不少空间，是顺序存储结构的一大缺点。

存储结构

```c
/* 二叉树的顺序存储表示 */
#define MAX_TREE_SIZE 100 /* 二叉树的最大节点数 */
typedef TElemType SqBiTree[MAX_TREE_SIZE]; /* 0号单元存储根节点 */

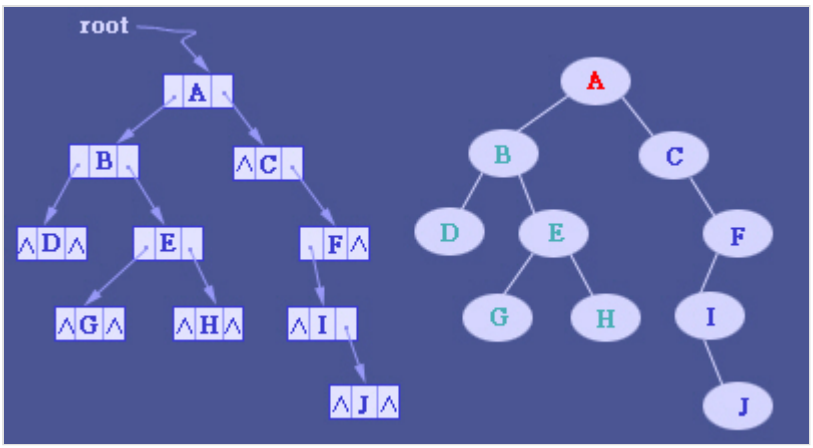
typedef struct
{
  int level,order; /* 即节点的层(按[满二叉树]计算) */
}position;
```

基本操作

基于C/C++的实现算法

二叉链表存储表示

在使用记录或存储器地址指针的程序设计语言中，二叉树通常用树结点结构来存储。有时也包含指向唯一的父节点的指针。如果一个结点的子结点个数小于2，一些子结点指针可能为空值，或者为特殊的哨兵结点。使用链表能避免顺序存储浪费空间的问题，算法和结构相对简单，但使用二叉链表，由于缺乏父链的指引，在找回父节点时需要重新扫描树得知父节点的节点地址。



基于链表的二叉树逻辑结构示意

存储结构

```
/* 二叉樹的二叉鏈表存儲表示 */
 typedef struct BiTNode
 {
   TElemType data;
   struct BiTNode *lchild,*rchild; /* 左右孩子指針 */
 }BiTNode,*BiTree;
```

基本操作

基于C/C++的实现算法

```
/* 二叉樹的二叉鏈表存儲的基本操作(22個) */
#define ClearBiTree DestroyBiTree /* 清空二叉樹和銷毀二叉樹的操作一樣 */
#include"func6-3.c"
/* 包括InitBiTree()、DestroyBiTree()、PreOrderTraverse()和InOrderTraverse()4函數 */
```

```
void CreateBiTree(BiTree *T)
{ /* 演算法6.4：按先序次序輸入二叉樹中結點的值(可為字元型或整型，在主程中定義)，*/
  /* 構造二叉鏈表表示的二叉樹T。變數Nil表示空(子)樹。有改動 */
  TElemType ch;
  scanf(form,&ch);
  if(ch==Nil) /* 空 */
    *T=NULL;
  else
  {
    *T=(BiTree)malloc(sizeof(BiTNode)); /* 生成根結點 */
    if(!*T)
      exit(OVERFLOW);
    (*T)->data=ch;
    CreateBiTree(&(*T)->lchild); /* 構造左子樹 */
    CreateBiTree(&(*T)->rchild); /* 構造右子樹 */
  }
}
```

```
Status BiTreeEmpty(BiTree T)
{ /* 初始條件：二叉樹T存在。操作結果：若T為空二叉樹，則返回TRUE，否則FALSE */
  if(T)
    return FALSE;
  else
    return TRUE;
}
```

```
int BiTreeDepth(BiTree T)
{ /* 初始條件：二叉樹T存在。操作結果：返回T的深度 */
  int i,j;
  if(T==NULL)  /*如果T=NULL,这样写便于理解，当然也可以写成if（!T）*/;
    return 0; /* 空樹深度為0 */
  if(T->lchild)
    i=BiTreeDepth(T->lchild); /* i為左子樹的深度 */
  else
    i=0;
  if(T->rchild)
    j=BiTreeDepth(T->rchild); /* j為右子樹的深度 */
  else
    j=0;
  return i>j?i+1:j+1; /* T的深度為其左右子樹的深度中的大者+1 */
}
```

```
TElemType Root(BiTree T)
{ /* 初始條件：二叉樹T存在。操作結果：返回T的根 */
  if(BiTreeEmpty(T))
    return Nil;
  else
    return T->data;
}
```

```
TElemType Value(BiTree p)
{ /* 初始條件：二叉樹T存在，p指向T中某個結點。操作結果：返回p所指結點的值 */
  return p->data;
}
```

```
void Assign(BiTree p,TElemType value)
{ /* 給p所指結點賦值為value */
  p->data=value;
}
```

```c
typedef BiTree QElemType; /* 設佇列元素為二叉樹的指針類型 */
#include"c3-2.h" /* 鏈佇列 */
#include"bo3-2.c" /* 鏈佇列的基本操作 */
TElemType Parent(BiTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：若e是T的非根結點，則返回它的雙親，否則返回＂空＂*/
  LinkQueue q;
  QElemType a;
  if(T) /* 非空樹 */
  {
    InitQueue(&q); /* 初始化佇列 */
    EnQueue(&q,T); /* 樹根指針入隊 */
    while(!QueueEmpty(q)) /* 隊不空 */
    {
      DeQueue(&q,&a); /* 出隊，佇列元素賦給a */
      if(a->lchild&&a->lchild->data==e||a->rchild&&a->rchild->data==e)
      /* 找到e(是其左或右孩子) */
        return a->data; /* 返回e的雙親的值 */
      else /* 沒找到e，則入隊其左右孩子指針(如果非空) */
      {
        if(a->lchild)
          EnQueue(&q,a->lchild);
        if(a->rchild)
          EnQueue(&q,a->rchild);
      }
    }
  }
  return Nil; /* 樹空或沒找到e */
}
```

```c
BiTree Point(BiTree T,TElemType s)
{ /* 返回二叉樹T中指向元素值為s的結點的指標。另加 */
  LinkQueue q;
  QElemType a;
  if(T) /* 非空樹 */
  {
    InitQueue(&q); /* 初始化佇列 */
    EnQueue(&q,T); /* 根指針入隊 */
    while(!QueueEmpty(q)) /* 隊不空 */
    {
      DeQueue(&q,&a); /* 出隊，佇列元素賦給a */
      if(a->data==s)
        return a;
      if(a->lchild) /* 有左孩子 */
        EnQueue(&q,a->lchild); /* 入隊左孩子 */
      if(a->rchild) /* 有右孩子 */
        EnQueue(&q,a->rchild); /* 入隊右孩子 */
    }
  }
  return NULL;
}
```

```c
TElemType LeftChild(BiTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點。操作結果：返回e的左孩子。若e無左孩子,則返回"空" */
  BiTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a->lchild) /* T中存在結點e且e存在左孩子 */
      return a->lchild->data; /* 返回e的左孩子的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType RightChild(BiTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點。操作結果：返回e的右孩子。若e無右孩子,則返回"空" */
  BiTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a->rchild) /* T中存在結點e且e存在右孩子 */
      return a->rchild->data; /* 返回e的右孩子的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType LeftSibling(BiTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：返回e的左兄弟。若e是T的左孩子或無左兄弟，則返回"空" */
  TElemType a;
  BiTree p;
  if(T) /* 非空樹 */
  {
    a=Parent(T,e); /* a為e的雙親 */
    if(a!=Nil) /* 找到e的雙親 */
    {
      p=Point(T,a); /* p為指向結點a的指標 */
      if(p->lchild&&p->rchild&&p->rchild->data==e) /* p存在左右孩子且右孩子是e */
        return p->lchild->data; /* 返回p的左孩子(e的左兄弟) */
    }
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType RightSibling(BiTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：返回e的右兄弟。若e是T的右孩子或無右兄弟，則返回"空" */
  TElemType a;
  BiTree p;
  if(T) /* 非空樹 */
  {
    a=Parent(T,e); /* a為e的雙親 */
    if(a!=Nil) /* 找到e的雙親 */
    {
      p=Point(T,a); /* p為指向結點a的指標 */
      if(p->lchild&&p->rchild&&p->lchild->data==e) /* p存在左右孩子且左孩子是e */
        return p->rchild->data; /* 返回p的右孩子(e的右兄弟) */
    }
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
Status InsertChild(BiTree p,int LR,BiTree c) /* 形參T無用 */
{ /* 初始條件：二叉樹T存在，p指向T中某個結點，LR為0或1，非空二叉樹c與T不相交且右子樹為空 */
  /* 操作結果：根據LR為0或1，插入c為T中p所指結點的左或右子樹。p所指結點的 */
  /*           原有左或右子樹則成為c的右子樹 */
  if(p) /* p不空 */
  {
    if(LR==0)
    {
      c->rchild=p->lchild;
      p->lchild=c;
    }
    else /* LR==1 */
    {
      c->rchild=p->rchild;
      p->rchild=c;
    }
    return OK;
  }
  return ERROR; /* p空 */
}
```

```
Status DeleteChild(BiTree p,int LR) /* 形參T無用 */
{ /* 初始條件：二叉樹T存在，p指向T中某個結點，LR為0或1 */
  /* 操作結果：根據LR為0或1，刪除T中p所指結點的左或右子樹 */
  if(p) /* p不空 */
  {
    if(LR==0) /* 刪除左子樹 */
      ClearBiTree(&p->lchild);
    else /* 刪除右子樹 */
      ClearBiTree(&p->rchild);
    return OK;
  }
  return ERROR; /* p空 */
}
```

```
typedef BiTree SElemType; /* 設棧元素為二叉樹的指針類型 */
#include"c3-1.h" /* 順序棧 */
#include"bo3-1.c" /* 順序棧的基本操作 */
void InOrderTraverse1(BiTree T,void(*Visit)(TElemType))
{ /* 採用二叉鏈表存儲結構，Visit是對資料元素操作的應用函數。演算法6.3，有改動 */
  /* 中序遍歷二叉樹T的非遞迴演算法(利用棧)，對每個資料元素調用函數Visit */
  SqStack S;
  InitStack(&S);
  while(T||!StackEmpty(S))
  {
    if(T)
    { /* 根指針進棧，遍歷左子樹 */
      Push(&S,T);
      T=T->lchild;
    }
    else
    { /* 根指針退棧，訪問根結點，遍歷右子樹 */
      Pop(&S,&T);
      Visit(T->data);
      T=T->rchild;
    }
  }
  printf("\n");
}
```

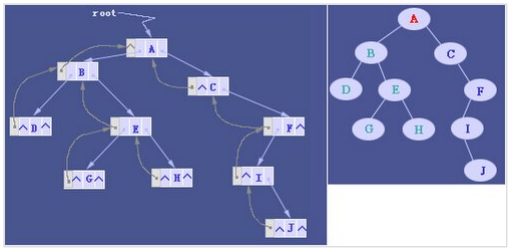
```
void InOrderTraverse2(BiTree T,void(*Visit)(TElemType))
{ /* 採用二叉鏈表存儲結構，Visit是對資料元素操作的應用函數。演算法6.2，有改動 */
  /* 中序遍歷二叉樹T的非遞迴演算法(利用棧)，對每個資料元素調用函數Visit */
  SqStack S;
  BiTree p;
  InitStack(&S);
  Push(&S,T); /* 根指針進棧 */
  while(!StackEmpty(S))
  {
    while(GetTop(S,&p)&&p)
      Push(&S,p->lchild); /* 向左走到盡頭 */
    Pop(&S,&p); /* 空指針退棧 */
    if(!StackEmpty(S))
    { /* 訪問結點，向右一步 */
      Pop(&S,&p);
      Visit(p->data);
      Push(&S,p->rchild);
    }
  }
  printf("\n");
}
```

```
void PostOrderTraverse(BiTree T,void(*Visit)(TElemType))
{ /* 初始條件：二叉樹T存在，Visit是對結點操作的應用函數 */
  /* 操作結果：後序遞迴遍歷T，對每個結點調用函數Visit一次且僅一次 */
  if(T) /* T不空 */
  {
    PostOrderTraverse(T->lchild,Visit); /* 先後序遍歷左子樹 */
    PostOrderTraverse(T->rchild,Visit); /* 再後序遍歷右子樹 */
    Visit(T->data); /* 最後訪問根結點 */
  }
}
```

```
void LevelOrderTraverse(BiTree T,void(*Visit)(TElemType))
{ /* 初始條件：二叉樹T存在，Visit是對結點操作的應用函數 */
  /* 操作結果：層序遞迴遍歷T(利用佇列)，對每個結點調用函數Visit一次且僅一次 */
  LinkQueue q;
  QElemType a;
  if(T)
  {
    InitQueue(&q); /* 初始化佇列q */
    EnQueue(&q,T); /* 根指針入隊 */
    while(!QueueEmpty(q)) /* 佇列不空 */
    {
      DeQueue(&q,&a); /* 出隊元素(指標),賦給a */
      Visit(a->data); /* 訪問a所指結點 */
      if(a->lchild!=NULL) /* a有左孩子 */
        EnQueue(&q,a->lchild); /* 入隊a的左孩子 */
      if(a->rchild!=NULL) /* a有右孩子 */
        EnQueue(&q,a->rchild); /* 入隊a的右孩子 */
    }
    printf("\n");
  }
}
```

三叉链表存储表示

改进于二叉链表，增加父节点的指引，能更好地实现节点间的访问，不过算法相对复杂。 当二叉树用三叉链表表示时，有N个结点，就会有N+2个空指针。



基于三叉链表的二叉树的逻辑结构

存储结构

```
/* 二叉樹的三叉鏈表存儲表示 */
typedef struct BiTPNode
{
  TElemType data;
  struct BiTPNode *parent,*lchild,*rchild; /* 父、左右孩子指針 */
}BiTPNode,*BiPTree;
```

基本操作

基于C/C++的实现算法

```
/* 二叉樹的三叉鏈表存儲的基本操作(21個) */
#define ClearBiTree DestroyBiTree /* 清空二叉樹和銷毀二叉樹的操作一樣 */
void InitBiTree(BiPTree *T)
{ /* 操作結果：構造空二叉樹T */
  *T=NULL;
}
```

```
void DestroyBiTree(BiPTree *T)
{ /* 初始條件：二叉樹T存在。操作結果：銷毀二叉樹T */
  if(*T) /* 非空樹 */
  {
    if((*T)->lchild) /* 有左孩子 */
      DestroyBiTree(&(*T)->lchild); /* 銷毀左孩子子樹 */
    if((*T)->rchild) /* 有右孩子 */
      DestroyBiTree(&(*T)->rchild); /* 銷毀右孩子子樹 */
    free(*T); /* 釋放根結點 */
    *T=NULL; /* 空指針賦0 */
  }
}
```

```
void CreateBiTree(BiPTree *T)
{ /* 按先序次序輸入二叉樹中結點的值(可為字元型或整型，在主程中定義)，*/
  /* 構造三叉鏈表表示的二叉樹T */
  TElemType ch;
  scanf(form,&ch);
  if(ch==Nil) /* 空 */
    *T=NULL;
  else
  {
    *T=(BiPTree)malloc(sizeof(BiTPNode)); /* 動態生成根結點 */
    if(!*T)
      exit(OVERFLOW);
    (*T)->data=ch; /* 給根結點賦值 */
    (*T)->parent=NULL; /* 根結點無雙親 */
    CreateBiTree(&(*T)->lchild); /* 構造左子樹 */
    if((*T)->lchild) /* 有左孩子 */
      (*T)->lchild->parent=*T; /* 給左孩子的雙親域賦值 */
    CreateBiTree(&(*T)->rchild); /* 構造右子樹 */
    if((*T)->rchild) /* 有右孩子 */
      (*T)->rchild->parent=*T; /* 給右孩子的雙親域賦值 */
  }
}
```

```
Status BiTreeEmpty(BiPTree T)
{ /* 初始條件：二叉樹T存在。操作結果：若T為空二叉樹，則返回TRUE，否則FALSE */
  if(T)
    return FALSE;
  else
    return TRUE;
}
```

```
int BiTreeDepth(BiPTree T)
{ /* 初始條件：二叉樹T存在。操作結果：返回T的深度 */
  int i,j;
  if(!T)
    return 0; /* 空樹深度為0 */
  if(T->lchild)
    i=BiTreeDepth(T->lchild); /* i為左子樹的深度 */
  else
    i=0;
  if(T->rchild)
    j=BiTreeDepth(T->rchild); /* j為右子樹的深度 */
  else
    j=0;
  return i>j?i+1:j+1; /* T的深度為其左右子樹的深度中的大者+1 */
}
```

```
TElemType Root(BiPTree T)
{ /* 初始條件：二叉樹T存在。操作結果：返回T的根 */
  if(T)
    return T->data;
  else
    return Nil;
}
```

```
TElemType Value(BiPTree p)
{ /* 初始條件：二叉樹T存在，p指向T中某個結點。操作結果：返回p所指結點的值 */
  return p->data;
}
```

```
void Assign(BiPTree p,TElemType value)
{ /* 給p所指結點賦值為value */
  p->data=value;
}
```

```
typedef BiPTree QElemType; /* 設佇列元素為二叉樹的指針類型 */
#include"c3-2.h" /* 鏈佇列 */
#include"bo3-2.c" /* 鏈佇列的基本操作 */
```

```
BiPTree Point(BiPTree T,TElemType e)
{ /* 返回二叉樹T中指向元素值為e的結點的指標。加 */
  LinkQueue q;
  QElemType a;
  if(T) /* 非空樹 */
  {
    InitQueue(&q); /* 初始化佇列 */
    EnQueue(&q,T); /* 根結點入隊 */
    while(!QueueEmpty(q)) /* 隊不空 */
    {
      DeQueue(&q,&a); /* 出隊，佇列元素賦給a */
      if(a->data==e)
        return a;
      if(a->lchild) /* 有左孩子 */
        EnQueue(&q,a->lchild); /* 入隊左孩子 */
      if(a->rchild) /* 有右孩子 */
        EnQueue(&q,a->rchild); /* 入隊右孩子 */
    }
  }
  return NULL;
}
```

```
TElemType Parent(BiPTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：若e是T的非根結點，則返回它的雙親，否則返回 " 空 "*/
  BiPTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a!=T) /* T中存在結點e且e是非根結點 */
      return a->parent->data; /* 返回e的雙親的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType LeftChild(BiPTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點。操作結果：返回e的左孩子。若e無左孩子,則返回" 空" */
  BiPTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a->lchild) /* T中存在結點e且e存在左孩子 */
      return a->lchild->data; /* 返回e的左孩子的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType RightChild(BiPTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點。操作結果：返回e的右孩子。若e無右孩子,則返回"空" */
  BiPTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a->rchild) /* T中存在結點e且e存在右孩子 */
      return a->rchild->data; /* 返回e的右孩子的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType LeftSibling(BiPTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：返回e的左兄弟。若e是T的左孩子或無左兄弟，則返回"空" */
  BiPTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a!=T&&a->parent->lchild&&a->parent->lchild!=a) /* T中存在結點e且e存在左兄弟 */
      return a->parent->lchild->data; /* 返回e的左兄弟的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
TElemType RightSibling(BiPTree T,TElemType e)
{ /* 初始條件：二叉樹T存在，e是T中某個結點 */
  /* 操作結果：返回e的右兄弟。若e是T的右孩子或無右兄弟，則返回"空" */
  BiPTree a;
  if(T) /* 非空樹 */
  {
    a=Point(T,e); /* a是結點e的指針 */
    if(a&&a!=T&&a->parent->rchild&&a->parent->rchild!=a) /* T中存在結點e且e存在右兄弟 */
      return a->parent->rchild->data; /* 返回e的右兄弟的值 */
  }
  return Nil; /* 其餘情況返回空 */
}
```

```
Status InsertChild(BiPTree p,int LR,BiPTree c) /* 形參T無用 */
{ /* 初始條件：二叉樹T存在，p指向T中某個結點，LR為0或1，非空二叉樹c與T不相交且右子樹為空 */
  /* 操作結果：根據LR為0或1，插入c為T中p所指結點的左或右子樹。p所指結點 */
  /*           的原有左或右子樹則成為c的右子樹 */
  if(p) /* p不空 */
  {
    if(LR==0)
    {
      c->rchild=p->lchild;
      if(c->rchild) /* c有右孩子(p原有左孩子) */
        c->rchild->parent=c;
      p->lchild=c;
      c->parent=p;
    }
    else /* LR==1 */
    {
      c->rchild=p->rchild;
      if(c->rchild) /* c有右孩子(p原有右孩子) */
        c->rchild->parent=c;
      p->rchild=c;
      c->parent=p;
    }
    return OK;
  }
  return ERROR; /* p空 */
}
```

```
Status DeleteChild(BiPTree p,int LR) /* 形參T無用 */
{ /* 初始條件：二叉樹T存在，p指向T中某個結點，LR為0或1 */
  /* 操作結果：根據LR為0或1，刪除T中p所指結點的左或右子樹 */
  if(p) /* p不空 */
  {
    if(LR==0) /* 刪除左子樹 */
      ClearBiTree(&p->lchild);
    else /* 刪除右子樹 */
      ClearBiTree(&p->rchild);
    return OK;
  }
  return ERROR; /* p空 */
}
```

```
void PreOrderTraverse(BiPTree T,void(*Visit)(BiPTree))
{ /* 先序遞迴遍歷二叉樹T */
  if(T)
  {
    Visit(T); /* 先訪問根結點 */
    PreOrderTraverse(T->lchild,Visit); /* 再先序遍歷左子樹 */
    PreOrderTraverse(T->rchild,Visit); /* 最後先序遍歷右子樹 */
  }
}
```

```
void InOrderTraverse(BiPTree T,void(*Visit)(BiPTree))
{ /* 中序遞迴遍歷二叉樹T */
  if(T)
  {
    InOrderTraverse(T->lchild,Visit); /* 中序遍歷左子樹 */
    Visit(T); /* 再訪問根結點 */
    InOrderTraverse(T->rchild,Visit); /* 最後中序遍歷右子樹 */
  }
}
```

```
void PostOrderTraverse(BiPTree T,void(*Visit)(BiPTree))
{ /* 後序遞迴遍歷二叉樹T */
  if(T)
  {
    PostOrderTraverse(T->lchild,Visit); /* 後序遍歷左子樹 */
    PostOrderTraverse(T->rchild,Visit); /* 後序遍歷右子樹 */
    Visit(T); /* 最後訪問根結點 */
  }
}
```

```
void LevelOrderTraverse(BiPTree T,void(*Visit)(BiPTree))
{ /* 層序遍歷二叉樹T(利用佇列) */
  LinkQueue q;
  QElemType a;
  if(T)
  {
    InitQueue(&q);
    EnQueue(&q,T);
    while(!QueueEmpty(q))
    {
      DeQueue(&q,&a);
      Visit(a);
      if(a->lchild!=NULL)
        EnQueue(&q,a->lchild);
      if(a->rchild!=NULL)
        EnQueue(&q,a->rchild);
    }
  }
}
```

遍历

我们经常希望访问树中的每一个结点并且查看它的值。有很多常见的顺序来访问所有的结点，而且每一种都有有用的性质。

深度优先遍历

在深度优先级中，我们希望从根结点访问最远的结点。和图的深度优先搜索不同的是，不需记住访问过的每一个结点，因为树中不会有环。前序，中序和后序遍历都是深度优先遍历的特例。

前（先）序、中序、后序遍历

遍历二叉树：L、D、R分别表示遍历左子树、访问根结点和遍历右子树，则先(根)序遍历二叉树的顺序是DLR，中(根)序遍历二叉树的顺序是LDR，后(根)序遍历二叉树的顺序是LRD。还有按层遍历二叉树。这些方法的时间复杂度都是O(n)，n为结点个数。

如果T2是由有序树T变换而来的二叉树，那么T中结点的前序就是T2中结点的前序，T中结点的后序就是T2中结点的中序。任何一棵二叉树的叶结点在先序、中序和后序遍历中的相对次序不发改变。设n,m为一棵二叉树上的两个结点，在中序遍历时，n在m前的条件是n在m的左方。前序序列和中序序列相同的二叉树为空树或任一结点均无左孩子的非空二叉树；中序序列和后序序列相同的二叉树为空树或任一结点均无右孩子的非空二叉树；前序序列和后序序列相同的二叉树为空树或仅有一个结点的二叉树。

假设我们有一个包含值的`value`和指向两个子结点的`left`和`right`的树结点结构。我们可以写出这样的过程：

```
visit(node)
    print node.value
    if node.left  != null then visit(node.left)
    if node.right != null then visit(node.right)
```

这样会用前序打印出树中的值。在前序，每个结点在访问它的子结点之前访问。类似地，如果打印语句在最后，每个结点在访问他的子节点之后访问，树中的值会用后序来打印。在这两种情况中，左子树中的值比右子树中得值先打印。

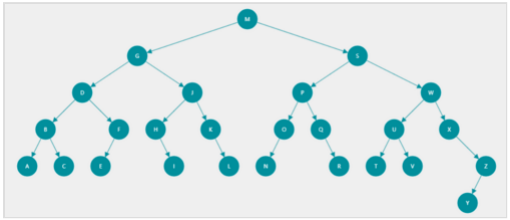
```
visit(node)
    if node.left  != null then visit(node.left)
    print node.value
    if node.right != null then visit(node.right)
```

最后，上面的中序遍历，每个结点在访问左子树和右子树之间访问。这在遍历二叉搜索树时很常用，因为它能用递增的顺序来遍历所有的值。

为什么呢？如果*n*是二叉搜索树的结点，那么*n*的左子树的所有结点的值都比n的值要小，而且*n*的右子树的所有节点的值都比n的值要大。因此，如果我们顺序遍历左子树，然后访问*n*，然后顺序遍历右子树。我们就已经循序访问了整个树。

后序遍历伪代码如下：

```
visit(node)
    if node.left  != null then visit(node.left)
    if node.right != null then visit(node.right)
    print node.value
```



BinaryTree

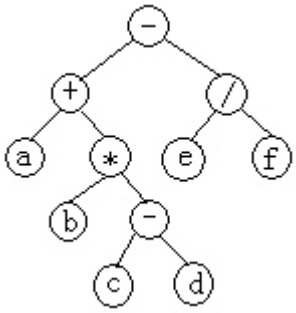
在这个二叉树中，

- 前序遍历的结果：M，G，D，B，A，C，F，E，J，H，I，K，L，S，P，O，N，Q，R，W，U，T，V，X，Z，Y
- 后序遍历的结果：A，C，B，E，F，D，I，H，L，K，J，G，N，O，R，Q，P，T，V，U，Y，Z，X，W，S，M
- 中序遍历的结果：A，B，C，D，E，F，G，H，I，J，K，L，M，N，O，P，Q，R，S，T，U，V，W，X，Y，Z

以上的递归算法使用与树的高度成比例的栈空间。如果我们在每个结点中存储指向父结点的指针，那样可以使用反复运算算法，只使用常数空间实现所有这些遍历。然而，指向父结点的指针占用更多的空间。这只在需要指向父节点的指针或栈空间有限时才使用。例如， 这是一个中序遍历的反复运算算法：

```
visit(root)
    prev    := null
    current := root
    next    := null

    while current != null
        if prev == current.parent
            prev := current
            next := current.left
        if next == null or prev == current.left
            print current.value
            prev := current
            next := current.right
        if next == null or prev == current.right
            prev := current
            next := current.parent
        current := next
```



用二叉树表示下述表达式：a+b*(c−d)−e/f

- 先序遍历的序列是：−+a*b−cd/ef
- 中序遍历的序列是：a+b*c−d−e/f
- 后序遍历的序列是：abcd−*+ef/−

广度优先遍历

和深度优先遍历不同，广度优先遍历会先访问离根节点最近的节点。二叉树的广度优先遍历又称按层次遍历。算法借助队列实现。

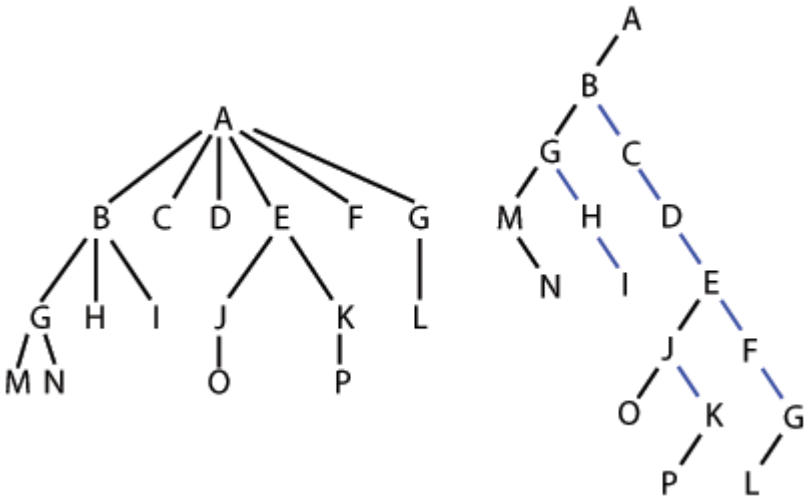
变换自多叉树

一般有序树可映射为二叉树，但反之未必成立。

n 叉树变换为二叉树的方法：二叉树中结点 x 的左子结点为 n 叉树中结点 x 的左子结点；二叉树中结点 x 的右子结点为 n 叉树中结点 x 的第一个右边的同级结点 y。

二叉树当且仅当根节点没有右子结点时可转换为 n 叉树。

例如，在左边的树中，A 有 6 个子结点{B,C,D,E,F,G}。它能被变换成右边的二叉树。



- 将一棵树变换为二叉树的方法：

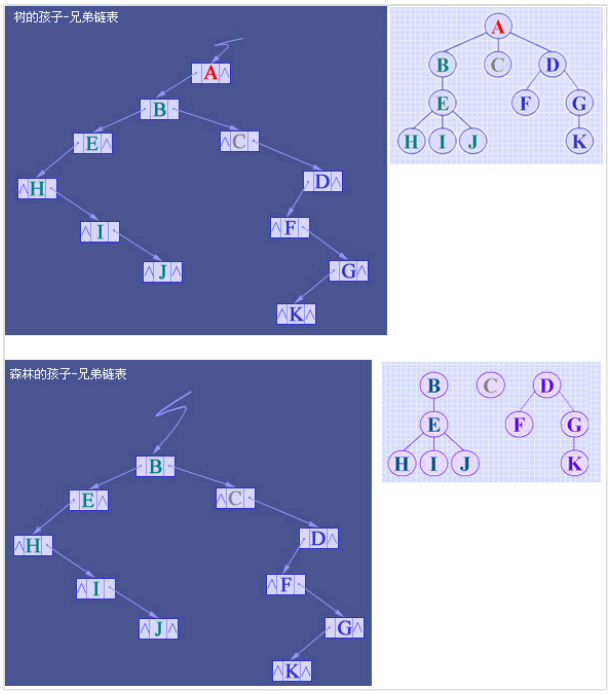
1. 在兄弟之间加一连线；
2. 对每个结点，除了其左孩子外，去除其与其余孩子之间的联系；
3. 以树的根结点为轴心，将整树顺时针转 45 度。

树的二叉链表标记法（孩子兄弟标记法）

树的二叉链表标记法（孩子兄弟标记法）是树和二叉树变换的介质。

存储结构

```
/* 樹的二叉鏈表(孩子–兄弟)存儲表示 */
typedef struct CSNode
{
  TElemType data;
  struct CSNode *firstchild,*nextsibling;
}CSNode,*CSTree;
```



二叉树与森林相互变换的逻辑示意

基本操作

基于C/C++的算法实现

线索二叉树

线索二叉树（英语：threaded binary tree，保留遍历时结点在任一序列的前驱和后继的信息）：若结点有左子树，则其lchild域指示其左孩子，否则令lchild域指示其前驱；若结点有右子树，则其rchild域指示其右孩子，否则令rchild指示其后继。还需在结点结构中增加两个标志域LTag和RTag。LTag=0时，lchild域指示结点的左孩子，LTag=1时，lchild域指示结点的前驱；RTag=0时，rchild域指示结点的右孩子，RTag=1时，rchild域指示结点的后继。以这种结点结构构成的二叉链表作为二叉树的存储结构，叫做线索链表，其中指向结点前驱和后继的指针叫做线索，加上线索的二叉树称为线索二叉树。对二叉树以某种次序遍历使其变为线索二叉树的过程叫做线索化。若对二叉树进行中序遍历，则所得的线索二叉树称为中序线索二叉树，线索链表称为为中序线索链表。线索二叉树是一种物理结构。

lchild	ltag	data	rtag	rchild

在中序线索树找结点后继的规律是：若其右标志为1，则右链为线索，指示其后继，否则遍历其右子树时访问的第一个结点（右子树最左下的结点）为其后继；找结点前驱的规律是：若其左标志为1，则左链为线索，指示其前驱，否则遍历左子树时最后访问的一个结点（左子树中最右下的结点）为其前驱。
在后序线索树中找到结点的后继分三种情况：

1. 若结点是二叉树的根，则其后继为空；
2. 若结点是其双亲的右孩子，或是其双亲的左孩子且其双亲没有右子树，则其后继即为双亲结点；
3. 若结点是其双亲的左孩子，且其双亲有右子树，则其后继为双亲右子树上按后序遍历列出的第一个结点。

二叉线索存储表示

存储结构

二叉树的二叉线索存储表示：在线索链表上添加一个头结点，并令其lchild域的指针指向二叉树的根结点，其rchild域的指针指向中序遍历时访问的最后一个结点。令二叉树中序序列中的第一个结点的lchild域指针和最后一个结点的rchild域的指针均指向头结点，这样就建立了一个双向线索链表。二叉树常采用二叉链表方式存储。

```
/* 二叉樹的二叉線索存儲表示 */
typedef enum{Link,Thread}PointerTag; /* Link(0)：指針，Thread(1)：線索 */
typedef struct BiThrNode
{
  TElemType data;
  struct BiThrNode *lchild,*rchild; /* 左右孩子指針 */
  PointerTag LTag,RTag; /* 左右標誌 */
}BiThrNode,*BiThrTree;
```

基本操作

基于C/C++的算法实现

外部链接

- binary trees (http://www.findstat.org/BinaryTrees)（页面存档备份 (https://web.archive.org/web/20200923083018/http://www.findstat.org/BinaryTrees)，存于互联网档案馆）
- entry in the FindStat (http://www.findstat.org/)（页面存档备份 (https://web.archive.org/web/20201031103908/http://www.findstat.org/)，存于互联网档案馆）数据库
- Gamedev.net introduction on binary trees (http://www.gamedev.net/page/resources/_/technical/general-programming/trees-part-2-binary-trees-r1433)（页面存档备份 (https://web.archive.org/web/20161218064824/http://www.gamedev.net/page/resources/_/technical/general-programming/trees-part-2-binary-trees-r1433)，存于互联网档案馆）
- 二叉树归纳证明 (http://www.brpreiss.com/books/opus4/html/page355.html)（页面存档备份 (https://web.archive.org/web/20190407044315/http://www.brpreiss.com/books/opus4/html/page355.html)，存于互联网档案馆）
- Balanced binary search tree on array How to create bottom-up an Ahnentafel list, or a balanced binary search tree on array (http://piergiu.wordpress.com/2010/02/21/balanced-binary-search-tree-on-array/)（页面存档备份 (https://web.archive.org/web/20190630092049/http://piergiu.wordpress.com/2010/02/21/balanced-binary-search-tree-on-array/)，存于互联网档案馆）
- Binary trees and Implementation of the same with working code examples (https://web.archive.org/web/20190408004155/http://www.cpphub.com/search/label/Binary%20trees)

1. Thomas H. Cormen; Charles E. Leiserson; Ronald L. Rivest; Clifford Stein. B.5.3 Binary and positional trees. Introduction to algorithms Third. Cambridge, Mass.: MIT Press. 2009: 1177–1178 [2023-02-06]. ISBN 978-0-262-03384-8. （原始内容存档于 2025-09-23）.
2. 高德纳. 计算机程序设计艺术 卷1：基本算法 第三版. Addison-Wesley. 1997: 401. ISBN 0-201-89683-4.

检索自“https://zh.wikipedia.org/w/index.php?title=二叉树&oldid=93918482”